

Taos: An Innovation in Operating Systems

With its ability to run heterogeneous processors in parallel, Taos should have a significant impact the world of parallel computing

DICK POUNTAIN

To describe Taos (pronounced "dowse," as in the Chinese philosophy Taoism) as an innovative operating system would be a profound understatement. This new operating system has so few similarities with operating systems like DOS that many observers, including myself, have initially had trouble believing in it.

Taos is strongly object-oriented and makes almost no distinction between data files and executable programs. It brings together many advanced ideas about persistent objects, object-oriented virtual memory, and data flow programming into one totally novel system.

It is also a parallel operating system that automatically configures itself and balances its own load to run on a network of distributed processors. Even more incredibly, it offers CPU transparency by running exactly the same code on different processors in a heterogeneous network (e.g., a mixture of Innos transputers, Intel 386 microprocessors, and Acorn RISC machines).

I must stress that Taos is not finished yet and that there are no commercial ports of it in use. Also, some of the radical features mentioned above had been only partially implemented at the time of this writing. Therefore, this article is more a preview of the concepts of Taos than a look at a finished product.

Taos is being developed by Tao Systems U.K. Ltd., a small firm that has existed for only about nine months and whose leading spokesmen are Nick Spicer and Chris Hinsley, two experienced programmers from different backgrounds. Spicer was a founding member of the European arm of Microway, the U.S. firm that specializes in high-performance floating-point hardware and, more currently, transputer products. Hinsley, on the other hand, is a veteran games programmer, an assembly language wizard who has been responsible for successful Atari ST

and Amiga games such as Verminator and Onslaught. Although writing exclusively in assembly language, Hinsley developed an object-oriented style for writing games based on extensive macro libraries, and this provided the original inspiration for Taos. Spicer added the parallel aspects to Taos, based on his experience with transputers and his dissatisfaction with Occam.

Running on a Virtual Processor

Perhaps the most important thing to understand about Taos, before even exploring how it works, is that it is designed to execute a virtual machine language called VP (for Virtual Processor). VP is the binary code of an imaginary RISC-



like processor with about 32 instructions.

To associate VP on a real processor, the company will provide a translator from VP into the native machine code of the target processor. The ingenious part is that the translation takes place at program load time, not during run time, as some erroneous reports have stated. Also, this is not an interpreted pseudocode as used in the UCSD P-System or PDP-11. When you load a program module onto a translator node, it gets turned into translator code before it is run, whereas if you load it onto a 386 microprocessor, it gets turned into Intel code.

Tao Systems is now committed to producing translators for the Innos T800 transputer, the Motorola 68000, the Pgc7600 (see "Beyond RISC: The Pgc7600 Microprocessor" on page 901S-109), the Intel 386/486, the Acorn RISC machines, and the Sun SPARC microprocessors. The first three of these have already been written.

The Taos program loader, called Alex, is part of the kernel that runs on each node in a parallel computer. Alex performs the native-code translation and optimization as it loads each program module. Because Taos is modular in nature and employs a novel late-binding scheme, the size of each module to be translated should be very small. In addition, since all the target processors are powerful CPUs, the time taken up in translation should be small.

Alex is not yet completed, and the prototype Taos systems that have been demonstrated thus far employ a conventional VP assembler to produce executable native code. Tao Systems seems confident that the design of the VP instruction set is such that writing the individual translators should not be too difficult, even for an-RISC-style processors such as the i486.

The first point to remember about Taos is that everything in it (apart from the native-code kernels at each node) consists of VP code, including all the applications. If Taos becomes established as an operating system, then compilers that translate from C and other languages into VP will undoubtedly begin to emerge.

To build a parallel Taos program, you just combine several tool modules that you can run on any combination of the supported target processors. You do not even need to know what the target processors are.

Object-Oriented Tools Everything in Taos is an object and, as in traditional object-oriented programming systems, objects contain both code and data. Under Taos, the code part is called a *sof*, which is equivalent to a method in conventional OOP systems. What would be considered data files under a conventional operating system are also objects. And each of these objects

has tools that can do various tasks, such as displaying the object's name or checking the access rights of a user.

All the executable code in Taos is contained in tools that must be fully relocatable, relocatable, and side-effect-free and that cannot, therefore, contain static data. Thus, Taos groups code and data together into objects but rigorously segregates the two within an object.

Tools are the smallest units of executable code under Taos, so you could think of them as being like subroutines, C functions, or Pascal procedures. However, tools are independently executable, so a better analogy would be with Forth, where you can always execute words directly without having to call them from a main program.

There are several different types of tools; the type depends on when they are loaded. *Permanence* tools are those that the boot system loads; these form the heart of the operating system. *Serviceperson* tools are loaded by a user-created configuration process at boot time; however, once loaded, they become part of the operating system and cannot be deactivated. These tools are roughly equivalent to the device drivers and extensions that CONFIG.SYS and AUTOEXEC.BAT load under DOS. *Application* tools are the user-executable programs of Taos. They can be virtual or nonvirtual. Virtual tools get loaded into memory only when the application calls them. When they cease to execute, they can be marked as freeable, and other tools can reclaim the space they occupy. Virtual tools are similar in concept to Borland's VROOM system and OS/2's dynamic link libraries. Nonvirtual tools are loaded with the application and remain in memory until the application is deactivated (i.e., quit). Virtual and nonvirtual tools are collectively referred to as *external* tools, and the applications and the user access them via their names, which resemble DOS or Unix path names. They are normally cached so that a subsequent access does not need to reload them from disk, unless memory is in short supply. Thus, an application in Taos consists largely of named references to external tools, allowing a great degree of reuse of code and keeping the individual executable units small.

In addition to external tools, Taos has *library* and *local* tools. Library tools are kept in named libraries, and many applications can share them; like external methods, they are cached for future use. Local tools are part of the actual code of an application and have single unique names, not path names. A local tool always overrides any external tool with the same name.

All Taos programs are started up by a *control* object, which is primarily a data object, a sort of template that contains all the information necessary to load and run the tools that make up the program and also has code that sets the loading of tools in motion. A control object roughly corresponds to a task or process in other operating systems. It contains a list of the path names of all its component tools, the stack space each one requires, and a bit mask, which defines the kind of messages it accepts. The permanent Taos kernel, which resides on each node of a parallel system, executes the control objects and loads and runs the necessary tools. Control objects can spawn children, and it is by this means that Taos programs distribute themselves around a parallel-processor network. Control objects can be inactivated without being removed from memory and can then be activated by other control objects that send them mail messages.

In the Taos tool system, all programs use late (i.e., load-time or run-time) binding. You will not have to link programs assembled or compiled under Taos, because linkage occurs when a program is run under the control of its control object. You can recompile a single tool without having to recompile the whole

BYTE ACTION SUMMARY

The Taos Operating System

A new operating system called Taos can run applications on several different types of processors at the same time, automatically balancing the load on each one. Taos also uses a consistent object-oriented design and a fast graphical user interface. Although Taos is not yet completed and is not compatible with existing software, it should have a significant impact on the parallel-computing industry.

application. This also means that you can design applications that load different tools according to the run-time environment they encounter.

Communicating by Mail

Taos is a message-passing operating system that uses mailboxes for communication between processes. All program I/O takes place through the mail system, as does the distribution of executable code. Taos automatically gives every control object a mailbox, and it can send or receive mail from any other object whose mailing address it knows. These will always include its own parents and children and any named resources, such as disk drives and displays. A control object can also send mail to

Taos is a message-passing operating system that uses mailboxes for communication between processes.

its siblings, if you take measures to record their addresses when you create them.

The kernel on each Taos node has a permanent Mail Guardian tool that handles all incoming mail for local objects on that node, all outgoing mail, and mail that is to be forwarded to another node.

The format of a Taos mail message includes a header that describes the size and type of the message. There are 16 predefined message types, including system, executable code, error data, and debugging data, and you can use numbered types of up to 255 to make private communications between two objects. A message type mask allows the message guardian to trap system messages, such as executable code destined for the kernel, and also lets the user objects prioritize the way the objects read their waiting mail.

The header also contains the addresses of the originator and the message destination, any other intended recipients for forwarding, and where to send a reply (if one is requested). As mentioned above, you can send mail to an inactivated object and reactivate it as a child of the sender object so that it can read the message and respond immediately. In other circumstances, such a message can just sit in the inactive object's mailbox until some other event activates it. If an inactive object gets removed due to a shortage of free memory, any pending mail is stored in the object's filefolder (i.e., written to disk; see below) and resent the next time the object is loaded. This might also happen when the Taos system is closed down so that messages, as well as objects, can be persistent.

Taos dynamically allocates the buffers that receive mail messages. In the event that there is not enough memory to create the buffer, the Message Guardian reads the header, discards the message data, and replies to the sender that the message has been trashed. The sender can then retry using a different route.

An Active Directory System

When Taos stores objects onto mass-storage media, such as hard disks, it uses the mailbox system and stores the objects into another type of object called a *filefolder*, which has some similarities to a DOS or Unix directory. But rather than being a passive storage structure, it is an active object (in fact, it's a control object). The tools attached to every filefolder are responsible for storing and retrieving objects from the folder and negotiating with the hardware device drivers that are necessary for this transfer. Because filefolders are actually instances of a broader category of Taos objects called *filters*, they can also process the data they transfer (e.g., compressing and expanding it transparently to the user).

I mentioned above that Taos tools have path names like those used to identify files in a hierarchical directory system such as DOS. Because Taos cannot permit the file duplication that DOS allows, path names are actually handled differently.

Where DOS relies on the *PATH* command and the user's knowledge of the directory tree and its contents, Taos must always locate objects automatically. The path name of a Taos object is the object name, rather than just a qualifying prefix indicating its current location. Therefore, Taos has built-in defaults related to the object type; by default, tools are expected to be in the tools filefolder. So if you request *mysstuff*, Taos will look for *tools/mysstuff*, unless you explicitly specify another filefolder. The path *tools/mysstuff* is actually a message to the control object tools requesting it to retrieve an object called *mysstuff*.

In large or multiuser systems, there can be multiple server devices and, therefore, multiple tools folders. In this case, Taos adds the mailbox ID of a master server to the path, which is roughly equivalent to a DOS drive; for example, */dtk01...server:/tools/mysstuff*. You must retrieve all future objects from this server until you explicitly change it.

Filters are control objects that receive input from a mailbox and a parameter string and send output to another mailbox. The destination address for the output is contained in the input message. The parameter string (which may be null) can tell the filter which of several actions to perform on its input. Filefolders are just one type of filter. In fact, all Taos applications constructed from TaosScript (which I'll describe later) commands are just pipelines of filters running in parallel and passing data from one to the other. Taos is a data flow operating system, extending the Unix pipe concept to its logical conclusion.

Parallelism Distributed Across a Network

Tao Systems is seeking a patent on the algorithm it uses to distribute parallel programs over a network of processors, so I can't describe it in detail here. It's basically a smarter enhancement of the flood-fill type of algorithm, which can take into account the loading of each processor node as measured by the number of processes running on it. An application that requires a large number of identical processes will get distributed in a manner that resembles water running down a mountain, where the flow seeks out the gullies or lowest points. Tao Systems claims that this mechanism achieves a high degree of automatic load balancing. Each node knows the current loading of all its nearest neighbors, and any control object can spawn children onto the neighbor that is least busy. This is done using a kernel call that invokes the mail system to transfer the executable VP code.

Deadlock due to circular network paths is avoided by an incrementally created routing table that remembers which paths have already been traversed. This is the part of the algorithm that is the subject of the patent, and such a table ought to work

BYTE's International Direct Response Postcards

From the most prestigious microcomputer magazine comes a new and inexpensive direct response medium—a postcard advertising insert bound into the highly respected International Section of BYTE Magazine, called *BYTE's International Direct Response Postcards*.

Postcard ads provide a direct-response vehicle that encourages prospects to respond to offers for information as well as direct sales. A postcard ad is versatile because it allows you to test your product to potential buyers. That's because you can use your card to sell computer products, generate leads, conduct market research or sell books and periodicals. Each postcard is perforated so it can be easily torn out and returned directly to you without any intermediary—guaranteeing leads that are current and "hot."

Expand your horizons into this valuable marketplace.

The Pan-European marketplace is preparing to open up to greatly expanded trade in 1992 with the breaking down of trade restrictions. And *BYTE's International Direct Response Postcards* presents the perfect opportunity to test your product in this marketplace and to get your company positioned in the minds of the major buyers.

These unique advertising postcards which will be inserted into the International Section will reach 70,000 influential BYTE paid readers throughout Europe.

In Europe, contact:

Roz Weyman (German-speaking)
at (01) 493-1451
McGraw-Hill Publications
34 Dover Street
London W1X 4BR
England

TAOS: AN INNOVATION IN OPERATING SYSTEMS

for any network topology. The routing table lets messages find their destination mailboxes. The routing algorithm can, hardware permitting, generate multiple paths between two nodes, which can be used as alternatives when mail gets blocked due to a shortage of buffer memory or a hardware failure.

When necessary, you can eschew the automatic mechanism and specify the particular node that you want an object to run on. You can also invoke a partially automatic mechanism in which you explicitly send several objects to a node but let Taos distribute them from that node. For example, you could specify that a 1000-process Mandelbrot calculation be run by sending 10 groups of 100 objects to different nodes of the network and letting Taos complete the distribution. The Taos kernel con-

**The Taos kernel contains
a time-slice scheduler that lets
many objects run
on the same processor.**

tains a time-slice scheduler that lets many objects run on the same processor.

The table lists the Taos-kernel calls (similar to DOS BIOS interrupts), and if you look at the entries under "control object management" whose names begin with OPEN, you will get some idea of how distribution is achieved.

From a programmer's point of view, remote interprocess communication is transparent under Taos. You only need to know the mailbox address of a remote object, and Taos will route messages to it without your having to specify the route. Taos Systems claims that near-optimal routings are usually achieved.

Language Dedicated to Pipelining

You interact with Taos via a job-control language edit/interpreter called TaoScribe, which processes a language called TaoScript. This is a data flow language that describes the flow of data through pipelines of tools specified by their paths. For example,

```
/data2 < proc2 < proc1 < /data1
```

would pass the data object data1 through the tools proc1 and proc2 and then store the output into data object data2. Both tools will be executed concurrently—possibly on different processors—and both will be started up before any data is transferred. In another example,

```
/hd/data2 /hd/data3 > proc3
```

both data objects would send data streams into the same tool. In the following,

```
/dtp_stuff/chapter2 "parameter" > tformat
```

A MESSAGE To OUR SUBSCRIBERS

FROM TIME TO TIME we make the BYTE subscriber list available to other companies who wish to send our subscribers material about their products. We take great care to screen these companies, choosing only those who are reputable, and whose products, services, or information we feel would be of interest to you. Direct mail is an efficient medium for presenting the latest personal computer goods and services to our subscribers.

Many BYTE subscribers appreciate this controlled use of our mailing list, and look forward to finding information of interest to them in the mail. Used are our subscribers' names and addresses only (no other information we may have is ever given).

While we believe the distribution of this information is of benefit to our subscribers, we firmly respect the wishes of any subscriber who does not want to receive such promotional literature. Should you wish to restrict the use of your name, simply send your request to the following address.

BYTE MAGAZINE

ATTN: SUBSCRIBER SERVICE
P.O. Box 535
HIGHTSTOWN, NJ 08520



TAOS: AN INNOVATION IN OPERATING SYSTEMS

TAOS-KERNEL CALLS

The TAOS-kernel calls, listed in functional groups. These calls are somewhat analogous to the BIOS system calls in DOS. The calls listed under "control object management" show how a parallel process might be distributed.

Memory management

ALLOCFAST	Allocate memory block from fast memory pool
ALLOCMIN	Allocate memory block given minimum size required
ALLOCMAX	Allocate memory block given maximum size required
FREEMEM	Free memory block

Object memory management

COPYNODE	Copy object
COPYHEAD	Copy object to head of linked list
COPYTAIL	Copy object to tail of linked list
DUMPLIST	Free memory of all nodes on a list
FREELIST	UNALEX and free memory of all nodes on a list

Mailbox management

SENDMAIL	Send mail message
COPYMAIL	Copy mail message and then send copy
READMAIL	Read mailbox

Control object management

STARTCONTROL	Start a control object locally
OPENCONTROL	Copy a control object and start locally
OPENCHILD	Distribute and start a control object in the network
CLOSECONTROL	Close a local control object
OPENARRAY	Distribute and open a number of control objects in the network
OPENFARM	Distribute and open multiple copies of a control object in the network
OPENDVICE	Transport a control object to a specified network node and start
OPENREMOTE	Transport a control object to a specified network node for distribution from that node and then start

Tool object management

VCALL	Virtual-call tool object
INQTOOL	Inquire if tool is available locally
OPENTOOL	Request tool-load
FLUSHPETL	Flush unreferenced tools from permanent and external tool list

General object management

VADDR	Obtain address of an embedded object
OBJPROC	Process an object in same thread
LISTPROC	Process a linked list of objects in same thread
LISTTEST	List inquiry for types of objects
LISTINFO	Inquire general-list information

Global-variable management

DECLARE	Declare named 64-bit integer value to child control objects
INQUIRE	Obtain 64-bit value for declared variable name from family tree
UNDECLARE	Locally remove last instance of a named variable declaration from declaration stack

Processor-type node identification

FINDTYPE	Inquire processor ID of processor node conforming to specified processor type and minimum memory requirement
----------	--

Local timer functions

GETTIME	Obtain local processor timer count
DELAY	Timed-deschedule

Template object management

ALEX	Convert template to Process Ready
UNALEX	Invalidate Process Ready status of object

The Cream.



The Crop.



There are plenty of places to get information in this industry. Too many. But if you want the best quality information, there's only one that rises to the top: **BYTEWEEK**, a weekly newsletter from the same professionals who produce **BYTE Magazine**.

Subscribe now and take advantage of a special subscription rate of \$395 (\$495 outside the U.S. and Canada). Don't miss this opportunity!

In the U.S. call **BYTEWEEK's** toll-free number: **1-800-258-5485**. In N.H. and outside the U.S. call **603-924-9281**.

BYTEWEEK offers a money-back guarantee if you are not completely satisfied.

BYTEWEEK 

One Plover Hill Lane
Dover, NH 03838

Taos: AN INNOVATION IN OPERATING SYSTEMS

the parameter string and the data are sent into a `tformat` tool.

The only operators in **TaoScript** are `/`, `<`, and `>`, but you can use these in combination to create forks and joins as well as linear pipes, although I found the resulting programs difficult to read. There are no control constructs, such as `IF...THEN` and `DO...WHILE`, in **TaoScript** because all control flow is governed inside the tools themselves by interpreting parameter strings. Thus, `[data2 "true", false] > proc3` might tell `proc3` to send its output to one of the objects `true` or `false`, according to the value of `/data2`.

You can save groups of **TaoScript** commands as named objects and use them as single commands, just like DOS batch files. The ordering of commands within a **TaoScript** program is often irrelevant since the tools are all running in parallel, and the data flow governs the order of their execution.

Objects in a GUI

There is also a simple graphical user interface built into **Taos**. Any control object can create a window object, which uses a buffer local to the object for maximum speed. These windows are sensitive to mouse events such as button clicks and dragging. Control objects don't need to worry about window movement or overlapping and refreshing, which are handled transparently by the GUI supervisor, but they can receive mouse `x,y` and time data from button clicks.

Only the Input Window can receive user input. The GUI indicates this window by a change of border color and brings it to the top of the pile. An object can send messages to its window to define two-dimensional screen areas as Area Event triggers; the GUI monitors such areas and sends an Area Event back to the object if the mouse pointer enters the area. Areas can range in size from 1 pixel to the whole window. The GUI can also monitor areas by sending a continuous stream of `x,y` data whenever the pointer is in the area. These simple facilities let you implement pop-up menus and dialog boxes.

Representative of the Next Generation

I have seen prototype versions of **Taos** running on five or six Intron transputers with a PC host. The GUI is lightning fast, and the demonstrations of multiple-windowed Mandelbrot plots are impressive. However, I have had no direct experience with programming under it, nor do I have complete documentation. Certain parts of **Taos**, including the Allen language translators, were not complete when this article went to press, and some aspects of the design are still undergoing evolution.

Tao Systems' claims for the efficiency of parallel-program distribution and message routing are strong and need to be independently tested by some parallel-computer vendors. There is no doubt, though, that **Taos** brings together many clever ideas in an elegant way. The philosophy it embodies is right on target for the next generation of operating systems, with its emphasis on persistent objects, data flow, and reusable modular tools. The VP system for supporting multiple processor types is so ingenious that it makes you wonder why other operating-system vendors have not tried it, until you reflect that it is only really viable for a starting-from-scratch operating system; the effort of rewriting all PC, Mac, or Unix software in VP would clearly be prohibitive.

Tao Systems is now busy demonstrating **Taos** to several big companies involved in parallel processing, including some from Japan. I shall be following the progress of **Taos** with great interest. ■

Dick Pommers is a contributing editor for BYTE based in London. He can be contacted on BIX as "dickp."